

DOCUMENTATION

Hybrid Monte Carlo implementation for the two-flavor Schwinger model

JFN Castellanos

We detail the implementation of the Hybrid Monte Carlo algorithm (HMC). We first discuss the implementation for pure gauge theory in Section 1. In Section 2 we introduce two degenerate fermions, and in Section 3 we write the resulting Dirac operator and force with the spin indices unrolled, which is the form used in the code. In Sections 4 and 5 we include the twisted mass and clover terms respectively.

1 HMC for the Schwinger model without fermions

We describe the Hybrid Monte Carlo algorithm for pure U(1) gauge theory. The general idea is to introduce a fictitious Hamiltonian $H[\pi, U]$, with momenta π , and to use the equations of motion

$$\frac{\partial H}{\partial \pi} = \frac{dU}{d\tau}, \quad \frac{\partial H}{\partial U} = -\frac{d\pi}{d\tau}, \quad (1)$$

to update the gauge configurations along a trajectory defined by the parameter τ . The Hamiltonian is given by

$$H = \frac{1}{2}\pi^2 + S[U], \quad (2)$$

where

$$S[U] = \beta \sum_{x \in V} \sum_{\mu < \nu} \text{Re}(1 - U_{\mu\nu}(x)), \quad \pi^2 = \sum_{x \in V} \sum_{\mu} \pi_{\mu}^2(x). \quad (3)$$

The momenta $\pi_{\mu}(x)$ are randomly generated with a normal distribution $p[\pi] \propto \exp(-\pi^2/2)$. The new configuration $[\pi', U']$ is accepted with probability $\min[1, \exp(-\Delta H)]$, where $\Delta H = H[\pi', U'] - H[\pi, U]$.

For the HMC to fulfill detailed balance, the solver for the Hamilton equations has to be *reversible* and to preserve the measure (check Ref. [1] for details regarding this statement). The former means that if the solver for eqs. (1) is described by a map $f[\pi, U] = [\pi', U']$, then

$$f[\pi, U] = [\pi', U'] \iff f[-\pi', U'] = [-\pi, U]. \quad (4)$$

Preserving the measure means that the Jacobian of f is equal to one. An integrator that satisfies both conditions is the *leapfrog algorithm*, which evolves $\pi_{\mu}(x)$ and $U_{\mu}(x)$ along a trajectory of length $N\Delta\tau$, $N \in \mathbb{N}$, in N steps of size $\Delta\tau$. For a classical mechanics problem with positions q and momenta p , the leapfrog method is described by¹

$$p\left(n + \frac{1}{2}\right) = p(n) + \frac{\Delta\tau}{2}F(n),$$

¹There are alternative ways of implementing the leapfrog algorithm.

$$\begin{aligned}
q(n+1) &= q(n) + \Delta\tau p\left(n + \frac{1}{2}\right), \\
p(n+1) &= p\left(n + \frac{1}{2}\right) + \frac{\Delta\tau}{2} F(n+1),
\end{aligned} \tag{5}$$

where n runs from 0 to $N-1$ and

$$F(p(n), q(n)) = F(n) = -\frac{\partial H}{\partial q}(n) = \frac{dp}{d\tau}(n). \tag{6}$$

Check Algorithm 1 for the pseudocode.

Algorithm 1 Leapfrog algorithm

```

 $\Delta\tau = \text{trajLength}/N$ 
 $p = p_0$  ▷ initial momentum
 $q = q_0$  ▷ initial position
for  $n = 0, \dots, N-1$  do
     $p \mathrel{+}= 0.5 \Delta\tau F(p, q)$ 
     $q \mathrel{+}= \Delta\tau p$ 
     $p \mathrel{+}= 0.5 \Delta\tau F(p, q)$ 
end for
return  $p, q$ 

```

In order to implement leapfrog for the U(1) theory, we first need the force $F = -\partial H/\partial U = -\partial S/\partial U$, i.e. for every $x \in V$ and every μ we have to compute

$$F(U_\mu(x)) = -\frac{\partial H}{\partial U_\mu}(x) = -\frac{\partial S}{\partial U_\mu}(x). \tag{7}$$

In group theory, we can compute the derivative with respect to a group element according to

$$\frac{df(U)}{dU} = \sum_{a=1}^{N_g} \frac{df(e^{i\omega T^a} U)}{d\omega} \Big|_{\omega=0}, \tag{8}$$

where the T^a are the N_g group generators. For U(1) this is just

$$\frac{df(U)}{dU} = \frac{df(e^{i\phi} U)}{d\phi} \Big|_{\phi=0}. \tag{9}$$

Differentiating the action results in

$$F[U_\mu(x)] = -\beta \operatorname{Im} [U_\mu(x) K_\mu^\dagger(x)], \tag{10}$$

where

$$K_\mu(x) = \sum_{\nu \neq \mu} \left[U_\nu(x) U_\mu(x + \hat{\nu}) U_\nu^\dagger(x + \hat{\mu}) + U_\nu^\dagger(x - \hat{\nu}) U_\mu(x - \hat{\nu}) U_\nu(x + \hat{\mu} - \hat{\nu}) \right] \tag{11}$$

is the sum of staples attached to the link $U_\mu(x)$.

When implementing the leapfrog algorithm in a gauge theory, we have to be careful to keep the link variables in the group. Therefore, we need a slight modification of eqs. (5): the additive

update of the coordinate is replaced by a multiplication with a group element. One way to do it is shown in Algorithm 2.

Algorithm 2 Leapfrog for U(1) theory

```

 $\Delta\tau = \text{trajLength}/N$ 
 $\pi = \pi_0$  ▷ random momentum sampled from a normal distribution
 $U = U_0$  ▷ initial gauge configuration
for  $n = 0, \dots, N - 1$  do
  Forces = Force( $U$ )
  for all  $x = 0, \dots, N_x - 1$ ;  $t = 0, \dots, N_t - 1$ ;  $\mu = 0, 1$  do
     $\pi[x, t, \mu] += 0.5 \Delta\tau \text{ Forces}[x, t, \mu]$ 
  end for
  for all  $x = 0, \dots, N_x - 1$ ;  $t = 0, \dots, N_t - 1$ ;  $\mu = 0, 1$  do
     $U[x, t, \mu] *= \exp(i\Delta\tau \pi[x, t, \mu])$ 
  end for
  Forces = Force( $U$ ) ▷ recompute with the updated links
  for all  $x = 0, \dots, N_x - 1$ ;  $t = 0, \dots, N_t - 1$ ;  $\mu = 0, 1$  do
     $\pi[x, t, \mu] += 0.5 \Delta\tau \text{ Forces}[x, t, \mu]$ 
  end for
end for
return  $U, \pi$ 

```

Note that the momentum half-steps and the link update have to be done in separate sweeps over the volume, and that the force must be recomputed in between: the second half-kick uses the force evaluated on the *updated* links, exactly as in Algorithm 1. Using the same force array for both half-kicks would break reversibility.

With this method, the HMC is implemented as follows:

- Generate a random momentum configuration $p[\pi] \propto \exp(-\pi^2/2)$.
- Use the leapfrog algorithm to evolve the configuration $[\pi, U] \rightarrow [\pi', U']$ along a trajectory of fixed length in N steps (usually $\text{trajLength} = 1$ and $N = 10$ works well, although this has to be tuned depending on the value of β).
- Accept or reject the new configuration $[\pi', U']$ with probability $\min[1, \exp(-\Delta H)]$, where $\Delta H = H[\pi', U'] - H[\pi, U]$. The acceptance rate depends strongly on the trajectory length and the number of steps.

These three steps are repeated to generate a set of configurations $[U]$. The momenta are randomly generated anew at the beginning of each trajectory.

2 Simulation of the 2-flavor Schwinger model

The Hybrid Monte Carlo algorithm for a U(1) gauge theory with fermions is essentially the same as in the quenched case: the leapfrog algorithm and the molecular dynamics step are not modified. However, computing the force and the action is more complicated. The Hamiltonian is given by

$$H = \frac{1}{2}\pi^2 + S[U, \psi, \psi^\dagger], \quad (12)$$

with the action

$$S[U, \psi, \psi^\dagger] = \beta \sum_{\mathbf{n}} \sum_{\mu < \nu} \text{Re}(1 - U_{\mu\nu}(\mathbf{n})) + \sum_{\mathbf{n}, \mathbf{n}'} \sum_{\alpha, \beta=0}^1 \psi_\alpha^\dagger(\mathbf{n}') D_{\mathbf{n}', \mathbf{n}}^{\alpha\beta} \psi_\beta(\mathbf{n}), \quad (13)$$

where D is a discretization of the Dirac operator. For instance, with Wilson fermions (in lattice units) we have

$$D[\mathbf{n}', \mathbf{n}]^{\alpha\beta} = (m_0 + 2) \delta^{\alpha\beta} \delta_{\mathbf{n}', \mathbf{n}} - \frac{1}{2} \sum_{\mu=0}^1 \left[(1 - \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{n}') \delta_{\mathbf{n}'+\hat{\mu}, \mathbf{n}} + (1 + \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{n}' - \hat{\mu}) \delta_{\mathbf{n}'-\hat{\mu}, \mathbf{n}} \right]. \quad (14)$$

The index $\mu = 0$ refers to time and $\mu = 1$ to space, and m_0 is the bare mass. We choose the following γ matrices,

$$\gamma_0 = \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \gamma_1 = \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \gamma_5 = i\sigma_1\sigma_2 = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}. \quad (15)$$

The partition function is

$$Z = \int \mathcal{D}[U] \mathcal{D}[\psi^\dagger] \mathcal{D}[\psi] e^{-S_G - S_F} = \int \mathcal{D}[U] e^{-S_G} \det(D), \quad (16)$$

where S_G corresponds to the gauge term and S_F to the fermions. The probability of a gauge configuration $[U]$ is therefore

$$P[U] = \frac{1}{Z} e^{-S_G[U]} \det(D), \quad (17)$$

which involves calculating the fermion determinant. This is computationally expensive. A way of replacing this step by the inversion of $DD^\dagger$ is the introduction of *pseudofermions*. These are complex scalar fields $\Phi = D\chi$, where χ is a vector whose real and imaginary parts are sampled independently from a normal distribution, so that $P[\chi] \propto \exp(-\chi^\dagger \chi)$. Their utility follows from the identity below, which holds for two degenerate flavors (check Ref. [1]):

$$\begin{aligned} \det(D_u) \det(D_d) &= \det(D) \det(D) = \det(D) \det(D^\dagger) \\ &= \det(DD^\dagger) \\ &= \int \mathcal{D}[\psi_u] \mathcal{D}[\psi_u^\dagger] \mathcal{D}[\psi_d] \mathcal{D}[\psi_d^\dagger] \exp\left(-\psi_u^\dagger D \psi_u - \psi_d^\dagger D \psi_d\right) \\ &= \pi^{-N} \int \mathcal{D}[\Phi_R] \mathcal{D}[\Phi_I] \exp\left(-\Phi^\dagger (DD^\dagger)^{-1} \Phi\right), \end{aligned} \quad (18)$$

where Φ_R and Φ_I denote the real and imaginary parts of Φ , and N is the number of complex components. Along the way we used γ_5 -hermiticity,

$$D^\dagger = \gamma_5 D \gamma_5 \quad \Rightarrow \quad \det(D^\dagger) = \det(\gamma_5 D \gamma_5) = \det(\gamma_5^2) \det(D) = \det(D). \quad (19)$$

Therefore, for two degenerate flavors we can work with the effective action

$$S[U, \Phi] = \beta \sum_{\mathbf{n}} \sum_{\mu < \nu} \text{Re}(1 - U_{\mu\nu}(\mathbf{n})) + \sum_{\mathbf{n}, \mathbf{n}'} \sum_{\alpha, \beta=0}^1 \Phi_\alpha^\dagger(\mathbf{n}') [(DD^\dagger)^{-1}]_{\mathbf{n}', \mathbf{n}}^{\alpha\beta} \Phi_\beta(\mathbf{n}), \quad (20)$$

which, together with property (18), gives an alternative to computing the determinant.

Matrix–vector products

The Dirac matrix for the Schwinger model has $(2N_x N_t)^2$ entries. Assembling the matrix is possible, but it might not be a good idea for large lattices. Instead, it is convenient to define the matrix operations

$$\begin{aligned}
 (D\psi)_\alpha(\mathbf{n}') &= \sum_{\mathbf{n}} \sum_{\beta=0}^1 D[\mathbf{n}', \mathbf{n}]^{\alpha\beta} \psi_\beta(\mathbf{n}) \\
 &= (m_0 + 2) \psi_\alpha(\mathbf{n}) \\
 &\quad - \frac{1}{2} \sum_{\beta=0}^1 \sum_{\mu=0}^1 \left[(1 - \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{n}') \psi_\beta(\mathbf{n}' + \hat{\mu}) + (1 + \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{n}' - \hat{\mu}) \psi_\beta(\mathbf{n}' - \hat{\mu}) \right] \quad (21)
 \end{aligned}$$

and

$$\begin{aligned}
 (D^\dagger \psi)_\alpha(\mathbf{n}') &= \sum_{\mathbf{n}} \sum_{\beta=0}^1 D^\dagger[\mathbf{n}', \mathbf{n}]^{\alpha\beta} \psi_\beta(\mathbf{n}) \\
 &= (m_0 + 2) \psi_\alpha(\mathbf{n}') \\
 &\quad - \frac{1}{2} \sum_{\beta=0}^1 \sum_{\mu=0}^1 \left[(1 - \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{n}' - \hat{\mu}) \psi_\beta(\mathbf{n}' - \hat{\mu}) + (1 + \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{n}') \psi_\beta(\mathbf{n}' + \hat{\mu}) \right]. \quad (22)
 \end{aligned}$$

The fields ψ_β have anti-periodic boundary conditions in the time direction, while the gauge links are periodic in every direction. This has to be taken into account when implementing eqs. (21) and (22).

Inversion of $DD^\dagger$

To invert the Hermitian matrix $DD^\dagger$, the simplest reliable method to implement is *conjugate gradient*. This approach works for Hermitian positive-definite matrices, which is the case for $DD^\dagger$. Close to the chiral limit the method becomes quite inefficient and takes many iterations to converge; in such cases better methods, like multigrid, are convenient. We stick to conjugate gradient for simplicity. For computing hadronic correlation functions the inverse of D (the propagator) is needed, and a suitable inversion method for that case is BiCGStab, a generalization of conjugate gradient.

We will not explain the intuition behind conjugate gradient here; we only list the steps needed to implement it. To solve $A\mathbf{x} = \mathbf{b}$:

- Propose an initial solution $\mathbf{x}_0$.
- Compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and define the initial search direction as $\mathbf{d}_0 = \mathbf{r}_0$.
- Iterate²

$$\begin{aligned}
 \alpha_i &= \frac{\langle \mathbf{r}_i, \mathbf{r}_i \rangle}{\langle \mathbf{d}_i, A\mathbf{d}_i \rangle}, & \mathbf{x}_{i+1} &= \mathbf{x}_i + \alpha_i \mathbf{d}_i, & \mathbf{r}_{i+1} &= \mathbf{r}_i - \alpha_i A\mathbf{d}_i, \\
 \beta_{i+1} &= \frac{\langle \mathbf{r}_{i+1}, \mathbf{r}_{i+1} \rangle}{\langle \mathbf{r}_i, \mathbf{r}_i \rangle}, & \mathbf{d}_{i+1} &= \mathbf{r}_{i+1} + \beta_{i+1} \mathbf{d}_i.
 \end{aligned}$$

²For complex vectors, the inner product is $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_i x_i y_i^*$.

- Repeat until $\text{err} = \langle \mathbf{r}_{i+1}, \mathbf{r}_{i+1} \rangle < \text{tol}$, where we are free to choose the tolerance.

For the simulation $A = DD^\dagger$. In order to avoid storing the matrix, we work with eqs. (21) and (22).

The fermion force

Now we proceed to compute the force, which is used in the leapfrog algorithm. We use the following definition for the derivative with respect to a link,

$$\frac{\partial f(U)}{\partial U} \equiv \frac{\partial f(e^{i\omega_\mu(\mathbf{z})}U)}{\partial \omega_\mu(\mathbf{z})} \bigg|_{\omega_\mu(\mathbf{z})=0}. \quad (23)$$

Then, using the effective action (20),

$$\begin{aligned} F_\mu(\mathbf{n}) &\equiv - \frac{\partial H(U, \pi)}{\partial \omega_\mu(\mathbf{n})} \bigg|_{\omega_\mu(\mathbf{n})=0} = - \frac{\partial S[U, \Phi]}{\partial \omega_\mu(\mathbf{n})} \bigg|_{\omega_\mu(\mathbf{n})=0} \\ &= F_\mu^{\text{gauge}}(\mathbf{n}) - \frac{\partial [\Phi^\dagger (DD^\dagger)^{-1} \Phi]}{\partial \omega_\mu(\mathbf{n})} \bigg|_{\omega_\mu(\mathbf{n})=0} \\ &= F_\mu^{\text{gauge}}(\mathbf{n}) - \Phi^\dagger \frac{\partial (DD^\dagger)^{-1}}{\partial \omega_\mu(\mathbf{n})} \bigg|_{\omega_\mu(\mathbf{n})=0} \Phi. \end{aligned} \quad (24)$$

To perform the derivative, consider $A = DD^\dagger$ and note that

$$\frac{\partial A^{-1}(x)}{\partial x} = \frac{\partial [A^{-1}(x) A(x) A^{-1}(x)]}{\partial x} = 2 \frac{\partial A^{-1}(x)}{\partial x} + A^{-1}(x) \frac{\partial A(x)}{\partial x} A^{-1}(x). \quad (25)$$

Thus

$$\frac{\partial A^{-1}(x)}{\partial x} = -A^{-1}(x) \frac{\partial A(x)}{\partial x} A^{-1}(x), \quad (26)$$

and therefore

$$\Phi^\dagger \frac{\partial (DD^\dagger)^{-1}}{\partial \omega_\mu(\mathbf{n})} \Phi = - \left(\left((DD^\dagger)^{-1} \Phi \right)^\dagger \left[\frac{\partial D}{\partial \omega_\mu(\mathbf{n})} D^\dagger + D \frac{\partial D^\dagger}{\partial \omega_\mu(\mathbf{n})} \right] \left(\left((DD^\dagger)^{-1} \Phi \right) \right). \quad (27)$$

This expression can be simplified. Define

$$M \equiv \frac{\partial D}{\partial \omega_\mu(\mathbf{n})} D^\dagger, \quad \text{so that} \quad M^\dagger = D \frac{\partial D^\dagger}{\partial \omega_\mu(\mathbf{n})}, \quad (28)$$

i.e. the bracket in eq. (27) is $M + M^\dagger$. For any vector v we have $v^\dagger M^\dagger v = (v^\dagger M v)^*$, hence $v^\dagger (M + M^\dagger) v = 2 \text{Re} [v^\dagger M v]$. With $v = (DD^\dagger)^{-1} \Phi$ this gives

$$\Phi^\dagger \frac{\partial (DD^\dagger)^{-1}}{\partial \omega_\mu(\mathbf{n})} \Phi = -2 \text{Re} \left[\left(\left((DD^\dagger)^{-1} \Phi \right)^\dagger \frac{\partial D}{\partial \omega_\mu(\mathbf{n})} D^\dagger \left(\left((DD^\dagger)^{-1} \Phi \right) \right) \right]. \quad (29)$$

Therefore, we only need one derivative,

$$\frac{\partial D[\mathbf{n}', \mathbf{n}]^{\alpha\beta}}{\partial \omega_\mu(\mathbf{z})} = \frac{\partial}{\partial \omega_\mu(\mathbf{z})} \left((m_0 + 2) \delta^{\alpha\beta} \delta_{\mathbf{n}', \mathbf{n}} \right)$$

$$\begin{aligned}
& -\frac{1}{2} \sum_{\nu=0}^1 \left[(1 - \gamma_\nu)^{\alpha\beta} e^{i\omega_\nu(\mathbf{n}')} U_\nu(\mathbf{n}') \delta_{\mathbf{n}'+\hat{\nu}, \mathbf{n}} \right. \\
& \quad \left. + (1 + \gamma_\nu)^{\alpha\beta} e^{-i\omega_\nu(\mathbf{n}'-\hat{\nu})} U_\nu^\dagger(\mathbf{n}'-\hat{\nu}) \delta_{\mathbf{n}'-\hat{\nu}, \mathbf{n}} \right] \Big|_{\omega_\mu(\mathbf{z})=0} \\
& = -\frac{i}{2} \left[(1 - \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{z}) \delta_{\mathbf{n}'+\hat{\mu}, \mathbf{n}} \delta_{\mathbf{n}', \mathbf{z}} \right. \\
& \quad \left. - (1 + \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{z}) \delta_{\mathbf{n}'-\hat{\mu}, \mathbf{n}} \delta_{\mathbf{n}, \mathbf{z}} \right]. \tag{30}
\end{aligned}$$

Multiplying from both sides by arbitrary fields we get

$$\begin{aligned}
\left(\psi^\dagger \frac{\partial D}{\partial \omega_\mu} \chi \right) (\mathbf{z}) &= \sum_{\mathbf{n}, \mathbf{n}'} \sum_{\alpha\beta} \psi_\alpha^\dagger(\mathbf{n}') \frac{\partial D[\mathbf{n}', \mathbf{n}]^{\alpha\beta}}{\partial \omega_\mu(\mathbf{z})} \chi_\beta(\mathbf{n}) \\
&= -\frac{i}{2} \sum_{\alpha\beta} \left[\psi_\alpha^\dagger(\mathbf{z}) (1 - \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{z}) \chi_\beta(\mathbf{z} + \hat{\mu}) - \psi_\alpha^\dagger(\mathbf{z} + \hat{\mu}) (1 + \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{z}) \chi_\beta(\mathbf{z}) \right]. \tag{31}
\end{aligned}$$

Combining eqs. (29) and (31), and using $\text{Re}[-iZ] = \text{Im}[Z]$, the force can be computed as

$$\begin{aligned}
F_\mu(\mathbf{n}) &= F_\mu^{\text{gauge}}(\mathbf{n}) \\
&+ \text{Im} \left(\sum_{\alpha\beta} \left[\psi_\alpha^\dagger(\mathbf{n}) (1 - \gamma_\mu)^{\alpha\beta} U_\mu(\mathbf{n}) \chi_\beta(\mathbf{n} + \hat{\mu}) - \psi_\alpha^\dagger(\mathbf{n} + \hat{\mu}) (1 + \gamma_\mu)^{\alpha\beta} U_\mu^\dagger(\mathbf{n}) \chi_\beta(\mathbf{n}) \right] \right), \tag{32}
\end{aligned}$$

where $\psi = (DD^\dagger)^{-1}\Phi$ and $\chi = D^\dagger\psi$.

With these ingredients the rest of the HMC is the same as in the pure gauge case.

3 Dirac operator implementation

With the γ matrices of eq. (15) the spin indices can be unrolled explicitly. This is the form actually coded, so we write eqs. (21), (22) and the second line of (32) component by component.

For $D\psi$ we obtain

$$\begin{aligned}
(D\psi)_0(\mathbf{n}) &= (m_0 + 2) \psi_0(\mathbf{n}) - \frac{1}{2} \left[U_0(\mathbf{n}) (\psi_0(\mathbf{n} + \hat{0}) - \psi_1(\mathbf{n} + \hat{0})) \right. \\
&\quad + U_1(\mathbf{n}) (\psi_0(\mathbf{n} + \hat{1}) + i \psi_1(\mathbf{n} + \hat{1})) \\
&\quad + U_0^\dagger(\mathbf{n} - \hat{0}) (\psi_0(\mathbf{n} - \hat{0}) + \psi_1(\mathbf{n} - \hat{0})) \\
&\quad \left. + U_1^\dagger(\mathbf{n} - \hat{1}) (\psi_0(\mathbf{n} - \hat{1}) - i \psi_1(\mathbf{n} - \hat{1})) \right], \tag{33}
\end{aligned}$$

$$\begin{aligned}
(D\psi)_1(\mathbf{n}) &= (m_0 + 2) \psi_1(\mathbf{n}) - \frac{1}{2} \left[U_0(\mathbf{n}) (-\psi_0(\mathbf{n} + \hat{0}) + \psi_1(\mathbf{n} + \hat{0})) \right. \\
&\quad + U_1(\mathbf{n}) (-i \psi_0(\mathbf{n} + \hat{1}) + \psi_1(\mathbf{n} + \hat{1})) \\
&\quad + U_0^\dagger(\mathbf{n} - \hat{0}) (\psi_0(\mathbf{n} - \hat{0}) + \psi_1(\mathbf{n} - \hat{0})) \\
&\quad \left. + U_1^\dagger(\mathbf{n} - \hat{1}) (i \psi_0(\mathbf{n} - \hat{1}) + \psi_1(\mathbf{n} - \hat{1})) \right]. \tag{34}
\end{aligned}$$

Similarly, for $D^\dagger \psi$,

$$\begin{aligned} (D^\dagger \psi)_0(\mathbf{n}) = (m_0 + 2) \psi_0(\mathbf{n}) - \frac{1}{2} \Big[& U_0^\dagger(\mathbf{n} - \hat{0}) (\psi_0(\mathbf{n} - \hat{0}) - \psi_1(\mathbf{n} - \hat{0})) \\ & + U_1^\dagger(\mathbf{n} - \hat{1}) (\psi_0(\mathbf{n} - \hat{1}) + i \psi_1(\mathbf{n} - \hat{1})) \\ & + U_0(\mathbf{n}) (\psi_0(\mathbf{n} + \hat{0}) + \psi_1(\mathbf{n} + \hat{0})) \\ & + U_1(\mathbf{n}) (\psi_0(\mathbf{n} + \hat{1}) - i \psi_1(\mathbf{n} + \hat{1})) \Big], \end{aligned} \quad (35)$$

$$\begin{aligned} (D^\dagger \psi)_1(\mathbf{n}) = (m_0 + 2) \psi_1(\mathbf{n}) - \frac{1}{2} \Big[& U_0^\dagger(\mathbf{n} - \hat{0}) (-\psi_0(\mathbf{n} - \hat{0}) + \psi_1(\mathbf{n} - \hat{0})) \\ & + U_1^\dagger(\mathbf{n} - \hat{1}) (-i \psi_0(\mathbf{n} - \hat{1}) + \psi_1(\mathbf{n} - \hat{1})) \\ & + U_0(\mathbf{n}) (\psi_0(\mathbf{n} + \hat{0}) + \psi_1(\mathbf{n} + \hat{0})) \\ & + U_1(\mathbf{n}) (i \psi_0(\mathbf{n} + \hat{1}) + \psi_1(\mathbf{n} + \hat{1})) \Big]. \end{aligned} \quad (36)$$

Finally, the fermionic part of the force reads

$$\begin{aligned} F_0(\mathbf{n}) - F_0^{\text{gauge}}(\mathbf{n}) = \text{Im} \Big[& U_0(\mathbf{n}) (\psi_0^\dagger(\mathbf{n}) - \psi_1^\dagger(\mathbf{n})) (\chi_0(\mathbf{n} + \hat{0}) - \chi_1(\mathbf{n} + \hat{0})) \\ & - U_0^\dagger(\mathbf{n}) (\psi_0^\dagger(\mathbf{n} + \hat{0}) + \psi_1^\dagger(\mathbf{n} + \hat{0})) (\chi_0(\mathbf{n}) + \chi_1(\mathbf{n})) \Big], \end{aligned} \quad (37)$$

$$\begin{aligned} F_1(\mathbf{n}) - F_1^{\text{gauge}}(\mathbf{n}) = \text{Im} \Big[& U_1(\mathbf{n}) (\psi_0^\dagger(\mathbf{n}) - i \psi_1^\dagger(\mathbf{n})) (\chi_0(\mathbf{n} + \hat{1}) + i \chi_1(\mathbf{n} + \hat{1})) \\ & - U_1^\dagger(\mathbf{n}) (\psi_0^\dagger(\mathbf{n} + \hat{1}) + i \psi_1^\dagger(\mathbf{n} + \hat{1})) (\chi_0(\mathbf{n}) - i \chi_1(\mathbf{n})) \Big]. \end{aligned} \quad (38)$$

4 Including the twisted mass term

The twisted mass term opens the eigenvalue spectrum of the Dirac matrix and reduces its condition number close to the critical mass. This term, however, mixes the two flavors,

$$D = D_W + i\mu_0 \gamma_5 \tau_3, \quad (39)$$

where μ_0 is the twisted mass parameter, D_W is the Wilson operator of eq. (14), τ_3 is the third Pauli matrix acting on flavor space, and

$$\gamma_5 = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}. \quad (40)$$

In principle the flavor mixing matrix could complicate things, but since we express most of our previous equations in terms of the product

$$DD^\dagger = D_W D_W^\dagger + \mu_0^2, \quad (41)$$

we only need to add μ_0^2 to the diagonal of our previous computation of $D_W D_W^\dagger$. Note that the following term inside the force is not modified:

$$\frac{\partial D}{\partial \omega_\mu(\mathbf{n})} D^\dagger + D \frac{\partial D^\dagger}{\partial \omega_\mu(\mathbf{n})} = \frac{\partial D_W}{\partial \omega_\mu(\mathbf{n})} D_W^\dagger + D_W \frac{\partial D_W^\dagger}{\partial \omega_\mu(\mathbf{n})}$$

$$+ \frac{\partial D_W}{\partial \omega_\mu(\mathbf{n})} (-i\gamma_5 \mu_0 \tau_3) + (i\gamma_5 \mu_0 \tau_3) \frac{\partial D_W^\dagger}{\partial \omega_\mu(\mathbf{n})}. \quad (42)$$

The two terms in the second line cancel: γ_5 -hermiticity of the Wilson operator, eq. (19), implies $\partial D_W^\dagger / \partial \omega_\mu = \gamma_5 (\partial D_W / \partial \omega_\mu) \gamma_5$, which follows from $\{\gamma_5, \gamma_\nu\} = 0$, and τ_3 commutes with everything in Dirac and coordinate space. The same argument, applied to eq. (41), shows that the cross terms there cancel as well.

5 Including the clover term

The clover term, which is added to the Dirac operator as $D = D_W + D_{\text{SW}}$, is defined by

$$D_{\text{SW}} = \frac{c_{\text{sw}}}{2} \sum_{\mu < \nu} \sigma_{\mu\nu} F_{\mu\nu}, \quad (43)$$

where

$$\sigma_{\mu\nu} = \frac{1}{2i} [\gamma_\mu, \gamma_\nu], \quad F_{\mu\nu} = -\frac{i}{8} (Q_{\mu\nu} - Q_{\mu\nu}^\dagger), \quad (44)$$

and

$$\begin{aligned} Q_{\mu\nu}(\mathbf{n}) = & U_\mu(\mathbf{n}) U_\nu(\mathbf{n} + \hat{\mu}) U_\mu^\dagger(\mathbf{n} + \hat{\nu}) U_\nu^\dagger(\mathbf{n}) \\ & + U_\nu(\mathbf{n}) U_\mu^\dagger(\mathbf{n} - \hat{\mu} + \hat{\nu}) U_\nu^\dagger(\mathbf{n} - \hat{\mu}) U_\mu(\mathbf{n} - \hat{\mu}) \\ & + U_\mu^\dagger(\mathbf{n} - \hat{\mu}) U_\nu^\dagger(\mathbf{n} - \hat{\mu} - \hat{\nu}) U_\mu(\mathbf{n} - \hat{\mu} - \hat{\nu}) U_\nu(\mathbf{n} - \hat{\nu}) \\ & + U_\nu^\dagger(\mathbf{n} - \hat{\nu}) U_\mu(\mathbf{n} - \hat{\nu}) U_\nu(\mathbf{n} + \hat{\mu} - \hat{\nu}) U_\mu^\dagger(\mathbf{n}). \end{aligned} \quad (45)$$

The function $Q_{\mu\nu}(\mathbf{n})$ is the sum over all plaquettes in the $\mu\nu$ plane touching the site $\mathbf{n}$, taken with counterclockwise orientation. Note that $Q_{\nu\mu} = Q_{\mu\nu}^\dagger$, so $F_{\mu\nu}$ is antisymmetric. Given that $\sigma_{\mu\nu} F_{\mu\nu}$ is therefore symmetric under $\mu \leftrightarrow \nu$, we can rewrite

$$\begin{aligned} D_{\text{SW}} &= \frac{c_{\text{sw}}}{4} \sum_{\mu, \nu=0}^1 \sigma_{\mu\nu} F_{\mu\nu} = \frac{c_{\text{sw}}}{8i} \sum_{\mu, \nu=0}^1 (\gamma_\mu \gamma_\nu F_{\mu\nu} - \gamma_\nu \gamma_\mu F_{\mu\nu}) = \frac{c_{\text{sw}}}{4i} \sum_{\mu, \nu=0}^1 \gamma_\mu \gamma_\nu F_{\mu\nu} \\ &= -\frac{c_{\text{sw}}}{32} \sum_{\mu, \nu=0}^1 \gamma_\mu \gamma_\nu (Q_{\mu\nu} - Q_{\mu\nu}^\dagger). \end{aligned} \quad (46)$$

In the Schwinger model this can be reduced further,

$$D_{\text{SW}} = -\frac{c_{\text{sw}}}{32} (\gamma_0 \gamma_1 - \gamma_1 \gamma_0) (Q_{01} - Q_{01}^\dagger), \quad (47)$$

and since

$$\gamma_0 \gamma_1 - \gamma_1 \gamma_0 = -2i\gamma_5, \quad (48)$$

we finally obtain

$$D_{\text{SW}}^{\alpha\beta}[\mathbf{n}', \mathbf{n}] = i \frac{c_{\text{sw}}}{16} (\gamma_5)_{\alpha\beta} (Q_{01}(\mathbf{n}) - Q_{01}^\dagger(\mathbf{n})) \delta_{\mathbf{n}, \mathbf{n}'}. \quad (49)$$

Clover contribution to the force

The clover term does modify the fermion force, because $Q_{\mu\nu}$ depends on the gauge links. Equation (29) remains valid, but we need the derivative of this new term. We use

$$\frac{\partial U_\nu(\mathbf{n})}{\partial \omega_\mu(\mathbf{z})} = i \delta_{\mu\nu} \delta_{\mathbf{n},\mathbf{z}} U_\nu(\mathbf{n}), \quad \frac{\partial U_\nu^\dagger(\mathbf{n})}{\partial \omega_\mu(\mathbf{z})} = -i \delta_{\mu\nu} \delta_{\mathbf{n},\mathbf{z}} U_\nu^\dagger(\mathbf{n}). \quad (50)$$

For convenience, we denote the four plaquettes entering $Q_{\mu\nu}$ as

$$\begin{aligned} P_{\mu\nu}^{(1)}(\mathbf{n}) &= U_\mu(\mathbf{n}) U_\nu(\mathbf{n} + \hat{\mu}) U_\mu^\dagger(\mathbf{n} + \hat{\nu}) U_\nu^\dagger(\mathbf{n}), \\ P_{\mu\nu}^{(2)}(\mathbf{n}) &= U_\nu(\mathbf{n}) U_\mu^\dagger(\mathbf{n} - \hat{\mu} + \hat{\nu}) U_\nu^\dagger(\mathbf{n} - \hat{\mu}) U_\mu(\mathbf{n} - \hat{\mu}), \\ P_{\mu\nu}^{(3)}(\mathbf{n}) &= U_\mu^\dagger(\mathbf{n} - \hat{\mu}) U_\nu^\dagger(\mathbf{n} - \hat{\mu} - \hat{\nu}) U_\mu(\mathbf{n} - \hat{\mu} - \hat{\nu}) U_\nu(\mathbf{n} - \hat{\nu}), \\ P_{\mu\nu}^{(4)}(\mathbf{n}) &= U_\nu^\dagger(\mathbf{n} - \hat{\nu}) U_\mu(\mathbf{n} - \hat{\nu}) U_\nu(\mathbf{n} + \hat{\mu} - \hat{\nu}) U_\mu^\dagger(\mathbf{n}), \end{aligned} \quad (51)$$

so that $Q_{\mu\nu} = \sum_{i=1}^4 P_{\mu\nu}^{(i)}$. We compute the derivative of the plaquettes with eqs. (50). For instance,

$$\frac{\partial P_{\mu\nu}^{(1)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} = i [\delta_{\rho\mu} \delta_{\mathbf{n},\mathbf{z}} + \delta_{\rho\nu} \delta_{\mathbf{n}+\hat{\mu},\mathbf{z}} - \delta_{\rho\mu} \delta_{\mathbf{n}+\hat{\nu},\mathbf{z}} - \delta_{\rho\nu} \delta_{\mathbf{n},\mathbf{z}}] P_{\mu\nu}^{(1)}(\mathbf{n}). \quad (52)$$

Since the bracket is real, the derivative of $P^{\dagger(1)}$ carries the opposite sign and the two combine into

$$\begin{aligned} \frac{\partial P_{\mu\nu}^{(1)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} - \frac{\partial P_{\mu\nu}^{\dagger(1)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} &= i [\delta_{\rho\mu} \delta_{\mathbf{n},\mathbf{z}} + \delta_{\rho\nu} \delta_{\mathbf{n}+\hat{\mu},\mathbf{z}} - \delta_{\rho\mu} \delta_{\mathbf{n}+\hat{\nu},\mathbf{z}} - \delta_{\rho\nu} \delta_{\mathbf{n},\mathbf{z}}] \left(P_{\mu\nu}^{(1)}(\mathbf{n}) + P_{\mu\nu}^{\dagger(1)}(\mathbf{n}) \right), \\ \frac{\partial P_{\mu\nu}^{(2)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} - \frac{\partial P_{\mu\nu}^{\dagger(2)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} &= i [\delta_{\rho\nu} \delta_{\mathbf{n},\mathbf{z}} - \delta_{\rho\mu} \delta_{\mathbf{n}-\hat{\mu}+\hat{\nu},\mathbf{z}} - \delta_{\rho\nu} \delta_{\mathbf{n}-\hat{\mu},\mathbf{z}} + \delta_{\rho\mu} \delta_{\mathbf{n}-\hat{\mu},\mathbf{z}}] \left(P_{\mu\nu}^{(2)}(\mathbf{n}) + P_{\mu\nu}^{\dagger(2)}(\mathbf{n}) \right), \\ \frac{\partial P_{\mu\nu}^{(3)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} - \frac{\partial P_{\mu\nu}^{\dagger(3)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} &= i [-\delta_{\rho\mu} \delta_{\mathbf{n}-\hat{\mu},\mathbf{z}} - \delta_{\rho\nu} \delta_{\mathbf{n}-\hat{\mu}-\hat{\nu},\mathbf{z}} + \delta_{\rho\mu} \delta_{\mathbf{n}-\hat{\mu}-\hat{\nu},\mathbf{z}} + \delta_{\rho\nu} \delta_{\mathbf{n}-\hat{\nu},\mathbf{z}}] \left(P_{\mu\nu}^{(3)}(\mathbf{n}) + P_{\mu\nu}^{\dagger(3)}(\mathbf{n}) \right), \\ \frac{\partial P_{\mu\nu}^{(4)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} - \frac{\partial P_{\mu\nu}^{\dagger(4)}(\mathbf{n})}{\partial \omega_\rho(\mathbf{z})} &= i [-\delta_{\rho\nu} \delta_{\mathbf{n}-\hat{\nu},\mathbf{z}} + \delta_{\rho\mu} \delta_{\mathbf{n}-\hat{\nu},\mathbf{z}} + \delta_{\rho\nu} \delta_{\mathbf{n}+\hat{\mu}-\hat{\nu},\mathbf{z}} - \delta_{\rho\mu} \delta_{\mathbf{n},\mathbf{z}}] \left(P_{\mu\nu}^{(4)}(\mathbf{n}) + P_{\mu\nu}^{\dagger(4)}(\mathbf{n}) \right). \end{aligned} \quad (53)$$

We need to compute

$$(F_{\text{SW}})_\rho(\mathbf{z}) = 2 \operatorname{Re} \left[\psi^\dagger \frac{\partial D_{\text{SW}}}{\partial \omega_\rho(\mathbf{z})} \chi \right], \quad (54)$$

with

$$\psi^\dagger = \left((DD^\dagger)^{-1} \Phi \right)^\dagger, \quad \chi = D^\dagger \left((DD^\dagger)^{-1} \Phi \right) \quad (55)$$

as before (D is here the full Dirac operator with all terms included). Inserting eq. (49),

$$\begin{aligned} (F_{\text{SW}})_\rho(\mathbf{z}) &= 2 \operatorname{Re} \left[\sum_{\mathbf{n},\mathbf{n}'} \sum_{\alpha\beta} \psi_\alpha^\dagger(\mathbf{n}) i \frac{c_{\text{sw}}}{16} (\gamma_5)_{\alpha\beta} \frac{\partial}{\partial \omega_\rho(\mathbf{z})} \left(Q_{01}(\mathbf{n}) - Q_{01}^\dagger(\mathbf{n}) \right) \delta_{\mathbf{n},\mathbf{n}'} \chi_\beta(\mathbf{n}') \right] \\ &= \operatorname{Re} \left[\sum_{\mathbf{n}} \left(\sum_{\alpha\beta} i \frac{c_{\text{sw}}}{8} \psi_\alpha^\dagger(\mathbf{n}) (\gamma_5)_{\alpha\beta} \chi_\beta(\mathbf{n}) \right) \frac{\partial}{\partial \omega_\rho(\mathbf{z})} \left(Q_{01}(\mathbf{n}) - Q_{01}^\dagger(\mathbf{n}) \right) \right]. \end{aligned} \quad (56)$$

We define

$$f(\mathbf{n}) := i \frac{c_{\text{sw}}}{8} \sum_{\alpha\beta} \psi_\alpha^\dagger(\mathbf{n}) (\gamma_5)_{\alpha\beta} \chi_\beta(\mathbf{n}) = i \frac{c_{\text{sw}}}{8} \left[\psi_1^\dagger(\mathbf{n}) \chi_1(\mathbf{n}) - \psi_0^\dagger(\mathbf{n}) \chi_0(\mathbf{n}) \right], \quad (57)$$

so that

$$(F_{\text{SW}})_\rho(\mathbf{z}) = \text{Re} \left[\sum_{\mathbf{n}} f(\mathbf{n}) \frac{\partial}{\partial \omega_\rho(\mathbf{z})} \left(Q_{01}(\mathbf{n}) - Q_{01}^\dagger(\mathbf{n}) \right) \right]. \quad (58)$$

Now we define

$$J^{(i)}(\mathbf{n}) = f(\mathbf{n}) \left(P_{01}^{(i)}(\mathbf{n}) + P_{01}^{\dagger(i)}(\mathbf{n}) \right), \quad (59)$$

and substitute eqs. (53) with $\mu = 0$, $\nu = 1$. Using $\text{Re}[iZ] = -\text{Im}[Z]$ we obtain the force

$$(F_{\text{SW}})_0(\mathbf{z}) = -\text{Im} \left[J^{(1)}(\mathbf{z}) - J^{(1)}(\mathbf{z} - \hat{1}) - J^{(2)}(\mathbf{z} + \hat{0} - \hat{1}) + J^{(2)}(\mathbf{z} + \hat{0}) \right. \\ \left. - J^{(3)}(\mathbf{z} + \hat{0}) + J^{(3)}(\mathbf{z} + \hat{0} + \hat{1}) + J^{(4)}(\mathbf{z} + \hat{1}) - J^{(4)}(\mathbf{z}) \right], \quad (60)$$

$$(F_{\text{SW}})_1(\mathbf{z}) = -\text{Im} \left[J^{(1)}(\mathbf{z} - \hat{0}) - J^{(1)}(\mathbf{z}) + J^{(2)}(\mathbf{z}) - J^{(2)}(\mathbf{z} + \hat{0}) \right. \\ \left. - J^{(3)}(\mathbf{z} + \hat{0} + \hat{1}) + J^{(3)}(\mathbf{z} + \hat{1}) - J^{(4)}(\mathbf{z} + \hat{1}) + J^{(4)}(\mathbf{z} - \hat{0} + \hat{1}) \right]. \quad (61)$$

When crossing the boundary in the time direction, the sign change of the fermion fields becomes irrelevant, because the two spinors entering $J^{(i)}$ are evaluated at the same site.

Including both the clover and the twisted mass term does not require any further modification, since

$$-\frac{\partial D_{\text{SW}}}{\partial \omega_\mu} i\mu_0 \gamma_5 \tau_3 + i\mu_0 \gamma_5 \tau_3 \frac{\partial D_{\text{SW}}^\dagger}{\partial \omega_\mu} = 0. \quad (62)$$

This can be proved using $D_{\text{SW}} = D_{\text{SW}}^\dagger$ and $[D_{\text{SW}}, \gamma_5] = 0$.

6 Automatic tuning of the integration step size

The HMC algorithm of Section 1 leaves two free parameters in the leapfrog integrator: the trajectory length $T \equiv N\Delta\tau$ and the number of steps N , with $\Delta\tau = T/N$. They play different roles. The step size $\Delta\tau$ controls the integration error and therefore the acceptance rate, while T controls how far the configuration is transported in molecular-dynamics time and therefore the autocorrelation of the observables. A good choice of $\Delta\tau$ cannot be read off the autocorrelations, and a good choice of T cannot be read off the acceptance rate.

This separation is what makes automation possible. We fix T by physics (the code takes it as the input parameter `trajectory_length`) and let the algorithm choose N so that a prescribed acceptance rate is reached. The implementation lives in `include/hmc_tuner.h` and runs during thermalization only.

6.1 Energy violation and acceptance rate

Let $\Delta H = H[\pi', U'] - H[\pi, U]$ be the energy violation of one trajectory. Because the accept/reject step of Section 1 leaves the target distribution invariant, the exact identity

$$\langle e^{-\Delta H} \rangle = 1 \quad (63)$$

holds for a correct implementation [3, 4]. Equation (63) is the sharpest single test of the force routines: any inconsistency between the force and the action violates it. If in addition ΔH is approximately Gaussian with variance σ^2 , then (63) forces

$$\langle \Delta H \rangle = \frac{\sigma^2}{2}, \quad P_{\text{acc}} = \langle \min[1, e^{-\Delta H}] \rangle = \text{erfc}\left(\sqrt{\frac{1}{2}\langle \Delta H \rangle}\right). \quad (64)$$

For a second-order integrator such as leapfrog the energy violation scales as

$$\langle \Delta H \rangle \propto V T \Delta\tau^4, \quad (65)$$

with V the lattice volume [3, 4]. Two consequences are worth recording. First, at fixed acceptance $\Delta\tau \propto V^{-1/4}$, so the tuned value from one volume gives a good starting guess at another. Second, (65) inverts into a one-step update rule: having measured $\langle \Delta H \rangle$ at some $\Delta\tau$, the step size reaching a target $\langle \Delta H \rangle_t$ (obtained via the solution to eq. (64) for a target P_{acc}) is

$$\Delta\tau' = \Delta\tau \left(\frac{\langle \Delta H \rangle_t}{\langle \Delta H \rangle} \right)^{1/4}. \quad (66)$$

This is implemented as `hmc::rescale_eps`. Because it uses the magnitude of ΔH rather than the binary accept/reject outcome, it converges in a few tens of trajectories. Its drawback is that it inherits the Gaussian assumption behind (64), which is only approximate: in test runs on a 32×32 lattice at $\beta = 4$, $m_0 = 0.2$, $c_{\text{sw}} = 1$ we measured $\langle \Delta H \rangle = 0.114(23)$, for which (64) predicts $P_{\text{acc}} = 0.74$ while the observed acceptance was 0.84. Equation (66) is therefore used only to obtain a starting value, not to set the final one.

Regarding the target itself, minimizing the cost per independent configuration gives an optimal acceptance of ≈ 0.65 under idealized assumptions [5]; in practice a somewhat higher value is safer, because near the integrator instability the distribution of ΔH develops a tail of large positive values. The code defaults to $\delta = 0.78$.

6.2 Dual averaging

Rather than relying on (65), the step size is tuned by stochastic optimization on $\log \Delta\tau$, using the dual-averaging scheme of Nesterov [6] in the form popularized for Hamiltonian Monte Carlo by Hoffman and Gelman [7]. This makes no assumption about how P_{acc} depends on $\Delta\tau$.

After the m -th trajectory of the tuning phase we form

$$a_m = \min[1, e^{-\Delta H_m}], \quad (67)$$

which is the Metropolis acceptance *probability* of that trajectory rather than the realized 0/1 outcome. Since $\langle a_m \rangle = P_{\text{acc}}$, this is an unbiased estimator of the same quantity with substantially smaller variance, and it is bounded, so a single large ΔH_m cannot destabilize the tuning. The updates are

$$\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - a_m), \quad (68)$$

$$\log \Delta\tau_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m, \quad (69)$$

$$\log \bar{\Delta}\tau_m = m^{-\kappa} \log \Delta\tau_m + (1 - m^{-\kappa}) \log \bar{\Delta}\tau_{m-1}, \quad (70)$$

with $\bar{H}_0 = 0$, $\mu = \log(10 \Delta\tau_0)$ and $\log \bar{\Delta\tau}_0 = \log \Delta\tau_0$. Equation (68) accumulates the signed error of the acceptance rate, (69) converts it into a step size, and (70) is a Polyak-Ruppert average that suppresses the noise of the individual iterates. We use the standard values $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$ [7].

Since T is held fixed and N must be an integer, the step size is quantized. The iterate (69) is kept as a continuous variable internally and converted on use through

$$N = \max[1, \text{round}(T/\Delta\tau)], \quad \Delta\tau \leftarrow T/N, \quad (71)$$

so that $N\Delta\tau = T$ holds exactly.

6.3 Warm-up schedule

Adaptation must stop before measurements begin. While $\Delta\tau$ is still changing, the transition probability changes from trajectory to trajectory, the chain is no longer Markov with respect to a single transition probability, and detailed balance does not hold [8]. Thermalization is therefore split into three phases, with N_{therm} the number of thermalization trajectories:

- $0 \leq i < 0.2 N_{\text{therm}}$ (*burn-in*). Adaptation is active. This phase is necessary: starting from a random configuration with an untuned $\Delta\tau$ the acceptance can be $O(10^{-4})$, so without adaptation the chain does not move at all and the configuration remains the initial hot one.
- $i = 0.2 N_{\text{therm}}$ (*reset*). The accumulators $\bar{H}$, m and $\log \bar{\Delta\tau}$ are discarded and re-initialized from the current $\Delta\tau$. The averaged step size (70) then reflects only equilibrated configurations, while the burn-in phase still supplied a workable $\Delta\tau$. This is `HMCTuner::reset`.
- $0.2 N_{\text{therm}} \leq i < 0.8 N_{\text{therm}}$ (*adaptation*), followed by $i \geq 0.8 N_{\text{therm}}$ (*frozen*). At $i = 0.8 N_{\text{therm}}$, `HMCTuner::freeze` sets $\Delta\tau = \exp(\log \bar{\Delta\tau})$ and stops all adaptation. The remaining trajectories run the exact kernel used for measurement, which allows the frozen acceptance rate to be verified before any data is collected.

References

- [1] C. Gattringer and C. B. Lang, *Quantum Chromodynamics on the Lattice: An Introductory Presentation*, Lect. Notes Phys. **788**, Springer (2010).
- [2] S. Duane, A. D. Kennedy, B. J. Pendleton and D. Roweth, *Hybrid Monte Carlo*, Phys. Lett. B **195** (1987) 216.
- [3] S. Gupta, A. Irbäck, F. Karsch and B. Petersson, *The acceptance probability in the hybrid Monte Carlo method*, Phys. Lett. B **242** (1990) 437.
- [4] A. D. Kennedy and B. Pendleton, *Acceptances and autocorrelations in hybrid Monte Carlo*, Nucl. Phys. B Proc. Suppl. **20** (1991) 118.
- [5] A. Beskos, N. Pillai, G. Roberts, J. M. Sanz-Serna and A. Stuart, *Optimal tuning of the hybrid Monte Carlo algorithm*, Bernoulli **19** (2013) 1501.
- [6] Y. Nesterov, *Primal-dual subgradient methods for convex problems*, Math. Program. **120** (2009) 221.
- [7] M. D. Hoffman and A. Gelman, *The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo*, J. Mach. Learn. Res. **15** (2014) 1593.

Algorithm 3 Step-size tuning during thermalization

```

1:  $\Delta\tau_0 = \text{trajLength}/N_0$ ,  $\mu = \log(10 \Delta\tau_0)$ ,  $\bar{H} = 0$ ,  $m = 0$ ,  $\log \bar{\Delta\tau} = \log \Delta\tau_0$ 
2:  $i_{\text{beg}} = 0.2 N_{\text{therm}}$ ,  $i_{\text{end}} = 0.8 N_{\text{therm}}$ 
3: for  $i = 0, \dots, N_{\text{therm}} - 1$  do
4:    $\Delta H \leftarrow \text{HMC\_UPDATE}()$  ▷ one trajectory with the current  $N$ 
5:   if  $i < i_{\text{end}}$  then
6:      $a = \min[1, e^{-\Delta H}]$ ,  $m += 1$ 
7:     update  $\bar{H}$ ,  $\log \Delta\tau$ ,  $\log \bar{\Delta\tau}$  by eqs. (68)–(70)
8:      $N = \max[1, \text{round}(\text{trajLength}/\Delta\tau)]$  ▷ eq. (71)
9:   end if
10:  if  $i = i_{\text{beg}}$  then
11:     $\bar{H} = 0$ ,  $m = 0$ ,  $\mu = \log(10 \Delta\tau)$ ,  $\log \bar{\Delta\tau} = \log \Delta\tau$  ▷ discard burn-in statistics
12:  end if
13:  if  $i = i_{\text{end}}$  then
14:     $\Delta\tau = \exp(\log \bar{\Delta\tau})$ ,  $N = \max[1, \text{round}(\text{trajLength}/\Delta\tau)]$ 
15:    freeze: no further adaptation
16:  end if
17: end for
18: return  $N$ 

```

[8] C. Andrieu and J. Thoms, *A tutorial on adaptive MCMC*, Stat. Comput. **18** (2008) 343.